Detecting Automated and Anomalous Editing Behaviour on Wikimedia at Scale

A Spark MLlib study of 11.0 million Wikimedia edits: bot-edit classification and editor-behaviour clustering

Mining of Massive Datasets (23CSE355)

End Semester Project Report

Team No: [Team No] Project Advisor: [Advisor Name]

[Reg No 1] [Name 1] · [Reg No 2] [Name 2] · [Reg No 3] [Name 3] · [Reg No 4] [Name 4]

Abstract

Wikimedia projects accept edits from anyone with no review before publication, absorbing on the order of 30 to 50 edits per second across all wikis. A large share of that traffic is automated. This project builds a distributed pipeline in Apache Spark over the complete all-time edit history of Simple English Wikipedia, comprising 15,201,370 dump records of which 11,023,987 are individual edits, and addresses two tasks from the same corpus: supervised classification of whether an edit was made by an automated account, and unsupervised clustering of editor behaviour. The dataset was selected specifically for label validity. Its bot indicator derives from MediaWiki user-group membership rather than from any property of the edit itself, which makes it genuinely learnable; the alternative candidate, GDELT 2.0, was rejected because its natural labels are deterministic lookup tables keyed on a single column. After cleaning, 10,987,077 edits remain, 23.7% of them automated. We engineer 35 behavioural features and reduce comment text to 24 dimensions with a truncated singular value decomposition computed on a distributed row matrix. Three classifiers and three clustering algorithms from Spark MLlib are compared under an actor-disjoint train/test split, so that every evaluated account is unseen during training. The best classifier, Random Forest on the full feature set, attains ROC-AUC 0.9759 and PR-AUC 0.9326. A paired ablation shows that structural behaviour alone is essentially sufficient: gradient-boosted trees reach ROC-AUC 0.9756 using no comment vocabulary whatsoever. The clustering result is reported as a negative one. Because bots constitute only 0.37% of accounts, cluster purity against the bot flag equals its own majority-class baseline exactly, and the informative measure, enrichment, shows bots merely 3.6 times over-represented in the high-activity cluster. Automation is therefore recoverable with supervision but is not the dominant axis of unsupervised behavioural variation.

1. Introduction

Wikimedia's editorial model is radically open: any visitor may change almost any page, and the change is live immediately. That openness is the reason the projects exist at the scale they do, and it is also the reason they require continuous, automated quality control. Across all Wikimedia projects the platform absorbs roughly 30 to 50 edits per second, a volume that no community of volunteer reviewers can inspect exhaustively.

A substantial and growing proportion of that traffic is not human. Approved maintenance bots perform indispensable work: fixing links, adding structured data, reverting obvious vandalism. Semi-automated tools such as AutoWikiBrowser and Huggle let a human drive edits at machine speed. And unapproved scripts operate outside policy altogether. Distinguishing these populations matters operationally, because reviewer attention is the scarcest resource in the system and should not be consumed by predictable machine edits, while unflagged automation is itself a policy violation and a known vector for abuse.

This report treats that operational need as two distinct data-mining problems over one corpus, in keeping with the requirement that the project implement both classification and clustering. The first problem is supervised: given everything observable about an edit at the moment it is made, predict whether its author is a flagged automated account. The second is unsupervised: without reference to any automation label, partition editors by behaviour and ask what populations actually exist.

We process the complete all-time history of Simple English Wikipedia using Apache Spark in local mode on a single twelve-core machine. The pipeline ingests a 951 MB compressed dump, converts it to a columnar store, cleans and labels it, engineers behavioural features, applies dimensionality reduction, and then trains and evaluates six classification configurations and twenty-one clustering configurations in Spark MLlib. All numerical results quoted in this report are read programmatically from the pipeline's own metric files, so that the report cannot diverge from the run that produced it.

2. Problem Statement

The project addresses two formally distinct tasks:

- **Task A — Edit-level automation classification.** Given a feature vector describing a single revision (its timing, size, namespace, the cadence of its author's recent activity, and the structure of its edit comment), predict the binary label label_bot, indicating whether the acting account is flagged as automated by MediaWiki. The operational framing is reviewer triage, so the quantity of interest is ranking quality under class imbalance rather than raw accuracy.
- **Task B — Editor-level behavioural segmentation.** Given an aggregated behavioural profile per account, partition editors into groups without using any automation label, then characterise the resulting groups and test externally whether they align with automation.

A secondary label, label_reverted, marking edits that were subsequently reverted, is derived and carried through the pipeline (14.1% of edits) and is used as an editor-level feature for Task B, but modelling it as a third task is left as future work.

2.1 Hypotheses stated in advance

To ensure the results section reports whether predictions held rather than narrating whatever the numbers happened to show, three hypotheses were committed to before modelling:

- **H1.** Cadence regularity, not activity volume, is the strongest single automation signal, because bots run on schedules whereas human editing is bursty.
- **H2.** Structural behavioural features alone will capture most of the achievable performance, with comment vocabulary adding a measurable but secondary increment.
- **H3.** Unsupervised behavioural clustering will partially, though not cleanly, recover the automation boundary, because semi-automated human tooling occupies the middle ground.

3. Literature Survey

The relevant literature falls into three threads: edit-level quality and abuse detection on Wikipedia, cross-platform automation detection by activity timing, and unsupervised identification of editor roles. A fourth, infrastructural thread supplies the distributed-computing background.

#	Work	Dataset	Features	Algorithms	Metrics	Contribution to this project
1	Halfaker & Taraborelli, ORES (Wikimedia, 2015)	Wikipedia revisions	revision metadata, diff size, user attributes	gradient boosting, random forest	precision/ recall at threshold, ROC-AUC	production framing: reviewer routing needs precision, not accuracy
2	Potthast, Stein & Gerling (ECIR, 2008)	940k English Wikipedia edits	character- and word-level diff features, comment text	logistic regression, rule baselines	precision, recall, F1	our comment-shape features (punctuation, caps, digit ratio)
3	Adler et al. (CICLing, 2011)	PAN-WVC-10	metadata + NLP + author reputation	random forest with feature-set ablation	ROC-AUC, PR-AUC	the paired structural-vs-full ablation design in Section 8
4	Davis et al., BotOrNot/Botometer (WWW, 2016)	Twitter accounts	temporal cadence, activity entropy, content	random forest	ROC-AUC, balanced accuracy	inter-event-gap regularity as the core automation signal
5	Chu et al. (IEEE TDSC, 2012)	500k Twitter accounts	inter-arrival timing, entropy, periodicity	entropy classifier, Bayesian	accuracy, false-positive rate	treating cadence variance rather than volume as discriminator
6	Welser et al. (iConference, 2011)	English Wikipedia editors	namespace distribution, edit mix	descriptive clustering	qualitative role validity	editor-archetype framing, namespace features
7	Yang et al. (ICWSM, 2016)	Wikipedia edit histories	per-editor activity profiles	k-means, mixture models	silhouette, interpretability	k selection by silhouette plus qualitative naming
8	Kumar, Spezzano & Subrahmanian, VEWS (KDD, 2015)	33k Wikipedia users	behavioural sequences, inter-edit time	SVM, random forest	accuracy, early-detection rate	per-editor rather than per-edit modelling as complement
9	Zaharia et al. (NSDI, 2012)	—	—	RDD abstraction	runtime scaling	in-memory distributed processing argument (Section 9)
1	Meng et al., MLlib (JMLR,	—	—	Spark MLlib	scalability	Pipeline API, distributed

#	Work	Dataset	Features	Algorithms	Metrics	Contribution to this project
0	2016)			pipelines	benchmarks	SVD, train-only transformer fitting

3.1 Synthesis

Works 1 to 3 and 8 establish that on Wikipedia, lightweight revision metadata combined with shallow comment features outperforms heavier natural-language processing once scale is taken into account, and that under class imbalance precision-recall analysis is more informative than accuracy. Work 3 in particular motivates our decision to train every model twice, with and without the text block, so that the contribution of vocabulary is measured rather than assumed.

Works 4 and 5 are drawn from a different platform deliberately. Both find that what distinguishes machine from human activity is not how much an account does but how regularly it does it. This is the direct motivation for our strongest engineered feature, the coefficient of variation of inter-edit gaps, and for hypothesis H1.

Works 6 and 7 provide the precedent for interpreting unsupervised editor clusters as social roles, and for validating the number of clusters by silhouette alongside human-readable characterisation. Our Section 8 follows that method but, as reported there, reaches a weaker conclusion than these works do.

4. Dataset Description

The data source is the Wikimedia Foundation's mediawiki_history dump, snapshot 2026-10, which publishes the complete edit history of every Wikimedia wiki as tab-separated values under a CC0 dedication. We use the simplewiki (Simple English Wikipedia) all-time file.

Property	Value
Source	dumps.wikimedia.org/other/mediawiki_history/2026-10/simplewiki/
Licence	CC0 public domain dedication
Compressed size	951 MB, bzip2-compressed TSV
Columns	78 tab-separated fields (the published documentation states 76)
Total dump records	15,201,370
— revision events	11,023,987 (the individual edits; our unit of analysis for Task A)
— page events	2,231,160
— user events	1,946,223
Columns retained	31 of 78, selected in Phase 0
Edits after cleaning	10,987,077 (0.33% removed)
Distinct editors	780,867
Primary label	label_bot — 2,598,870 positive (23.65%)
Secondary label	label_reverted — 1,550,915 positive (14.12%)
Live companion feed	stream.wikimedia.org/v2/stream/recentchange (server-sent events)

4.1 Justification of dataset selection

The decisive criterion was label validity. The field event_user_is_bot_by is populated from MediaWiki user-group membership and the bot-name registry, which are administrative facts about an account, not properties of the individual revision. Predicting it from edit behaviour is therefore a genuine inference problem.

The obvious alternative, GDELT 2.0, was evaluated and rejected on exactly this criterion. Its natural supervised targets, QuadClass and GoldsteinScale, are both deterministic lookup tables keyed on the single

EventCode column. Retaining that column lets a classifier score almost perfectly by reproducing a lookup table, which demonstrates nothing about the data; removing it discards the signal and leaves the task ill-posed. Choosing a dataset whose label is not a function of its own features is, in our view, the most consequential decision in the project.

Three further considerations supported the choice: the corpus is genuinely large and the dominant workload is a wide shuffle rather than a scan (Section 9); both required task families are supported from one corpus, with a second supervised label available; and the data requires no credentials and is accompanied by a live stream suitable for demonstration.

4.2 Dataset understanding: the schema is not as documented

The Wikitech documentation for these dumps states that the dataset contains 76 fields. The 2026-10 snapshot in fact ships **78** tab-separated columns; the two additional fields are event_user_central_id and user_central_id, which the documentation has not yet incorporated. Accepting the documented field order on trust would have misaligned every column in the file.

Column order was therefore established empirically. We profiled every column index over a sample of real records, recording its fill rate and value shape (boolean, integer, timestamp, free text, or base-36 hash), and matched those signatures against the documented field groups. Reliable anchors were wiki_db (constant), event_entity (only three distinct values), page_id, revision_text_sha1 (40-character base-36 hashes) and revision_tags (final column). Counting between anchors confirmed that the event_user_* block occupies exactly columns 7 to 28, which matches the documented group once the two additional identifier fields are included.

One column remains undocumented. Column 2 holds a large integer and is populated almost exclusively on records where event_entity is user. It corresponds to neither user_id nor user_central_id, both of which appear separately in the same records. We label it event_log_id and flag it as undocumented; it is unused, because every downstream phase filters to revision events, where it is always empty.

4.3 Parsing hazards

- **Embedded quotation marks.** The event_comment field contains raw double-quote characters. Spark's CSV reader treats the double quote as a quoting character by default and will absorb tab delimiters inside an apparently quoted region, silently shifting every subsequent column on the affected record. Quote processing must be disabled outright.
- **No header row.** The first line of the file is data.
- **Nulls are empty strings**, not a sentinel such as \N.
- **Non-ISO timestamps** of the form 2001-05-18 21:57:10.0, requiring the explicit pattern yyyy-MM-dd HH:mm:ss.S.
- **bzip2 is splittable** under Hadoop, so Spark parallelises the read across tasks. Had the dump been gzip-compressed, the entire 951 MB would have been confined to a single task.

5. Data Preprocessing

Preprocessing is implemented in Spark DataFrames and Spark SQL. Phase 0 converts the compressed TSV into Snappy-compressed Parquet, retaining 31 of the 78 columns; Phase 3 performs the cleaning proper. Every field is read as a string and cast explicitly, because allowing Spark to infer types silently drops records given the dump's empty-string nulls and non-ISO timestamps.

5.1 Recovering deleted-page revisions

The first substantive cleaning decision concerns missing page metadata. When a page is deleted, MediaWiki nulls the current-state page_* columns but retains their *_historical counterparts. In our data 14.8% of revisions have a null page_namespace for this reason.

Discarding those records would have removed roughly one edit in seven. More seriously, it would have introduced a systematic bias rather than a mere loss of volume: deleted pages are precisely where vandalism concentrates, so dropping them would skew the distribution of the label_reverted target. We therefore coalesce each current-state field with its historical counterpart before any null filtering. The effect is substantial — rows lost to null structural keys fall from approximately 15% to 0.33%.

src/p03_prepare.py

```
df = (df.withColumn("page_namespace",
        F.coalesce("page_namespace", "page_namespace_historical"))
      .withColumn("page_title",
        F.coalesce("page_title", "page_title_historical")))
```

5.2 Duplicates and missing values

De-duplication uses revision_id, the natural key for an edit; the all-time dump can repeat a revision where a page was deleted and later restored. In this snapshot 0 duplicate records were found, and 36,910 records were removed for lacking a structural key after historical recovery.

Preprocessing step	Records
Revision records read	11,023,987
Removed: duplicate revision_id	0
Removed: null structural key	36,910
Clean records	**10,987,077**
Automated edits (label_bot = 1)	2,598,870 (23.65%)
Reverted edits (label_reverted = 1)	1,550,915 (14.12%)

Imputation is applied only where absence carries a definite meaning, and the choice of filler is deliberate. A null revision_text_bytes_diff occurs on page creation, where the correct value is the full text size. A null event_user_seconds_since_previous_revision marks a user's first edit; this is filled with the sentinel −1 rather than 0, because 0 would assert an instantaneous repeat edit and actively mislead the model. A companion binary feature records whether a previous edit existed, so the sentinel remains distinguishable after scaling.

5.3 A null that becomes a silent NaN

Boolean fields required particular care, and getting this wrong produced the most instructive failure of the project. In SQL, comparing a null to a literal yields null, not false, so a naive cast of the form col == "true" leaves nulls intact. Those nulls survive into VectorAssembler, where they become NaN; StandardScaler then propagates the NaN, and Spark finally rejects the feature vector with a message naming neither the column nor the phase responsible. The failure surfaces deep inside a model fit, far from its cause.

page_is_redirect is null for 9.8% of records, again because of page deletion, which made this failure certain rather than hypothetical. The fix coalesces each boolean cast to false, which is the defensible reading of "the current-state flag is unknown". More importantly, the pipeline now asserts a null contract: after labelling, Phase 3 checks every column whose value reaches a feature vector and aborts with the offending column names if any null remains. Three columns are deliberately exempt and documented as such — event_user_id (anonymous edits have none, and it only builds the grouping key), revision_parent_id (nullity is itself the page-creation feature), and revision_seconds_to_identity_revert (null means never reverted, and it is unused).

src/p03_prepare.py

```
for c in BOOL_COLS:
    df = df.withColumn(c,
        F.coalesce(F.col(c) == F.lit("true"), F.lit(False)))
```

```
    # fail loudly and name the column, rather than NaN-ing into a model fit
    bad = {c: v for c, v in remaining.items() if v}
    if bad:
        raise SystemExit(f"FATAL: nulls remain in feature inputs: {bad}")
```

5.4 Label definitions

label_bot is 1 where event_user_is_bot_by is non-empty, indicating the acting account is flagged automated by group membership or by the bot-name registry. label_reverted is 1 where revision_is_identity_reverted is true. Neither is a deterministic function of any engineered feature, which is the property that makes the supervised task meaningful.

6. Feature Engineering

Two feature matrices are produced. The edit-level matrix carries one row per revision with 35 numeric features, one categorical field (namespace) and a 24-dimension text block, and drives Task A. The editor-level matrix carries one row per account with 19 aggregated features and drives Task B.

Group	Features	Rationale
Temporal	hour, day of week, weekend, night, hour sine/cosine	scheduled processes show a flat 24-hour profile; humans sleep. Cyclic encoding avoids treating hour 23 and hour 0 as maximally distant
Edit size	byte size, signed diff, absolute diff, log absolute diff, diff sign, diff ratio	maintenance automation makes small, uniform, often additive changes
Cadence	log seconds since author's previous edit, has-previous flag, log author edit count, log page revision count	machine timing is regular; human timing is bursty (H1)
Account	account age in days at edit time, is-anonymous	separates IP editors, new accounts and long-lived tool accounts
Page context	namespace, is-content-namespace, is-redirect, is-minor, is-page-creation	bots concentrate in File and Template namespaces
Comment shape	length, word count, section-marker presence, wikilink count, URL count, digit/uppercase/punctuation ratios, is-empty	captures the structure of the message without using its vocabulary
Revision tags	tag count, mobile, visual editor, undo	tags naming automation are excluded to prevent leakage
Editor-level	gap coefficient of variation, night/weekend ratio, minor ratio, content ratio, revert ratio, edits per active day, pages per edit, namespace count, activity span	behavioural profile for unsupervised segmentation

The feature intended to carry the most weight under H1 is e_gap_cv, the coefficient of variation of an account's inter-edit intervals, computed with a window function partitioned by actor and ordered by timestamp. A perfectly scheduled process has a coefficient near zero; human editing sessions produce large values.

6.1 Dimensionality reduction by truncated SVD

Edit comments are tokenised, hashed into 2048 TF-IDF buckets with a minimum document frequency of 5, and reduced to 24 dimensions by a truncated singular value decomposition computed on a distributed RowMatrix. Only the right singular matrix V, of shape 2048 by 24, is collected to the driver and broadcast; the projection then runs independently on each partition.

The decomposition is fitted on a 4% sample of records rather than the full corpus. The right singular vectors converge with far fewer rows than are available, and fitting on 15% of 11 million records took 21.5 minutes against a few minutes at 4%; the projection is nonetheless applied to all 10,987,077 rows. The leading singular values are 1746, 1373, 1176, 1067, 980, 921.

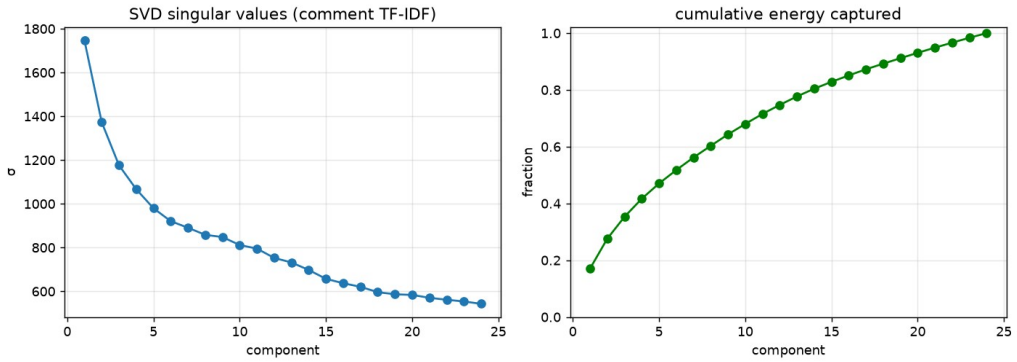


Figure 1. Singular value spectrum of the comment TF-IDF matrix (left) and cumulative energy captured by successive components (right). The spectrum decays smoothly rather than exhibiting a sharp elbow, indicating that comment vocabulary has no small set of dominant directions; 24 components were retained.

We use SVD rather than PCA deliberately: PCA centres the data, which would destroy the sparsity of a TF-IDF matrix and make the distributed computation substantially more expensive for no analytical gain.

7. MLlib Model Implementation

All models are built as Spark ML Pipeline objects, so that every fitted transformer is estimated from training data only. Three classifiers are compared for Task A and three clustering algorithms for Task B.

7.1 Leakage controls

Three separate leakage channels had to be closed before any reported score could be trusted. We consider this the methodological core of the project.

Channel	Why it leaks	Control applied
Account identity	Usernames literally contain the substring "Bot" (for example SchlurcherBot), and event_user_groups contains the very bot group that defines the label	Both excluded from every feature vector. The username is used only as a grouping key for aggregation, which never reaches a model
Random row split	The same high-volume bot account would appear in both train and test, letting the model memorise account-specific cadence and edit-count fingerprints and report a wildly optimistic score	Train/test split by hashed actor key, so test accounts are never seen in training. The script asserts the intersection is empty and prints the count
Scaler fitted on all data	StandardScaler means and variances would encode statistics of the test set	The scaler is a stage inside the Pipeline, so it is fitted on the training fold alone

Comment text is retained as a feature, with the reasoning stated explicitly rather than left implicit. It is observable at the moment the edit is made and is not derived from user-group membership, so it is a legitimate if strong predictor. To keep that claim testable, every classifier is trained twice: once on a structural feature set containing no comment vocabulary, and once on the full set including the SVD block. The difference between the two quantifies the contribution of text instead of assuming it.

7.2 The actor-disjoint split

The split is deterministic: the actor key is hashed with the random seed, reduced modulo 100, and bucketed. This yields 8,619,472 training rows and 2,367,605 test rows with **zero shared actors**, verified at run time.

src/p05_classification.py

```
def actor_split(df, train_fraction, seed):
    bucket = F.abs(F.hash(F.concat(F.col("actor_key"),
                F.lit(str(seed))))) % 100
    cut = int(train_fraction * 100)
    return (df.filter(F.col("_bucket") < cut).drop("_bucket"),
```

```
    df.filter(F.col("_bucket") >= cut).drop("_bucket"))
```

A consequence worth noting is that the class balance differs between folds — 25.6% positive in training against 16.7% in test. This is inherent to actor-disjoint splitting on a corpus where a small number of accounts own a large share of edits: whichever fold receives the highest-volume bots dominates its own positive rate. The split remains correct and the evaluation honest; the asymmetry simply reflects the true structure of the data.

7.3 Model configurations

Task	Algorithm	Configuration
Classification	Logistic Regression	maxIter 50, regParam 0.01, L2 (elasticNetParam 0)
Classification	Random Forest	40 trees, maxDepth 10, maxBins 48
Classification	Gradient-Boosted Trees	maxIter 30, maxDepth 5, maxBins 48
Clustering	K-Means	k from 2 to 8, maxIter 50
Clustering	Bisecting K-Means	k from 2 to 8, maxIter 30
Clustering	Gaussian Mixture	k from 2 to 8, maxIter 40

For Task B the automation flag is deliberately excluded from the input matrix and retained only for external validation afterwards. Editors with fewer than 5 edits are excluded, leaving 108,954 of 780,867 accounts: single-edit accounts dominate the long tail, have no inter-edit interval at all, and would otherwise collapse into one uninformative mega-cluster.

One defensive measure proved necessary. Requesting k clusters does not guarantee k non-empty ones, and ClusteringEvaluator raises an assertion failure when a fit degenerates, which would abort the entire sweep. The implementation detects fewer than two non-empty clusters, records the degeneracy, and continues.

8. Experimental Results

8.1 Classification

All six configurations were evaluated on the full held-out set of 2,367,605 edits from unseen accounts. The starred row is the best model by ROC-AUC.

Model	Features	ROC-AUC	PR-AUC	Accuracy	F1 (wtd)	Bot prec.	Bot recall	Bot F1
Logistic Regression	structural	0.9114	0.6792	0.8489	0.8558	0.5375	0.6840	0.6020
Random Forest	structural	0.9713	0.8948	0.9219	0.9133	0.9522	0.5605	0.7057
GBT	structural	0.9756	0.9098	0.9564	0.9565	0.8648	0.8760	0.8703
Logistic Regression	full	0.9512	0.9122	0.9605	0.9594	0.9348	0.8207	0.8741
Random Forest *	full	0.9759	0.9326	0.9667	0.9655	0.9806	0.8171	0.8914
GBT	full	0.9655	0.9144	0.9609	0.9599	0.9356	0.8229	0.8756

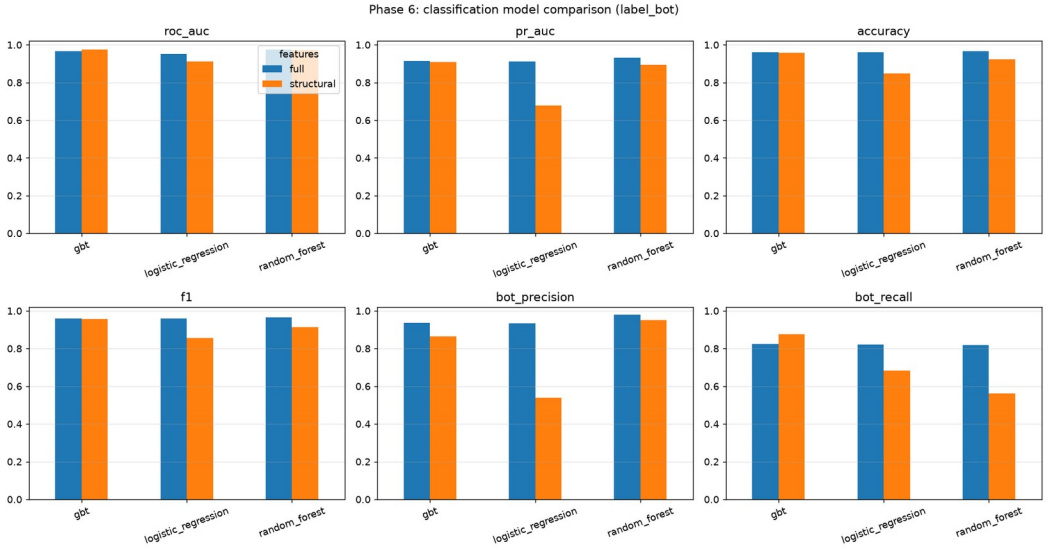


Figure 2. Six performance metrics across the three classifiers, with structural and full feature sets side by side. The gap between the two bars in each pair is the measured contribution of comment text.

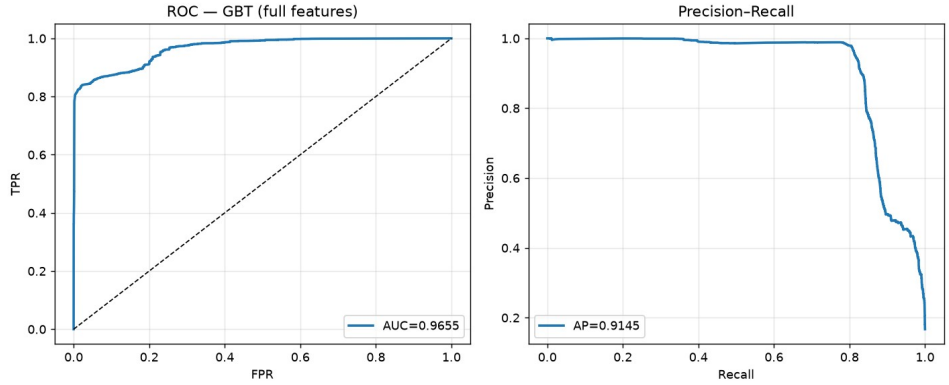


Figure 3. ROC and precision-recall curves for gradient-boosted trees on the full feature set. Under class imbalance the precision-recall view is the more informative of the two, because it ignores the large true-negative mass that inflates ROC.

The best configuration is Random Forest on the full feature set, with ROC-AUC 0.9759, PR-AUC 0.9326, accuracy 0.9667 and a bot-class F1 of 0.8914 at the default threshold. Bot-class precision is 0.9806 against recall 0.8171, which is the correct trade-off for the reviewer-triage framing: a flagged edit is almost always genuinely automated.

8.2 The structural-versus-full ablation

H2 held, and more strongly than anticipated. Gradient-boosted trees reach ROC-AUC 0.9756 using no comment vocabulary at all — within 0.03 percentage points of the best model overall. Adding the text block to GBT does not improve it but slightly reduces it, to 0.9655. Automation is legible from behaviour rather than from what an edit says.

Text matters, but only to the linear model, where it is transformative: logistic regression improves from ROC-AUC 0.9114 to 0.9512 and, far more dramatically, from PR-AUC 0.6792 to 0.9122. The tree ensembles gain comparatively little (random forest 0.9713 to 0.9759). The natural interpretation is that the ensembles already capture, through axis-aligned splits and their interactions, the structure that the SVD dimensions merely make linearly separable. The ablation therefore measures model capacity as much as it measures feature value — a caveat we would not have detected had we trained each model only once.

8.3 Clustering

The full sweep of 21 configurations is given below. Silhouette is computed with squared Euclidean distance on the standardised editor matrix.

Algorithm	k	Silhouette	Non-empty	Cluster sizes
kmeans	2	0.1780	2	61,186, 47,768
kmeans	3	0.1311	3	60,075, 30,350, 18,529
kmeans	4	0.1162	4	47,742, 32,775, 16,310, 12,127
kmeans	5	0.0851	5	39,329, 26,559, 16,012, 14,908, ...
kmeans	6	0.1325	6	35,077, 21,911, 21,604, 15,437, ...
kmeans	7	0.1786	7	40,715, 22,362, 16,161, 15,383, ...
kmeans	8	0.1495	8	29,293, 19,527, 19,038, 15,340, ...
bisecting_kmeans	2	degenerate	1	108,954
bisecting_kmeans	3	0.3406	2	82,951, 26,003
bisecting_kmeans	4	0.1031	3	58,013, 26,003, 24,938
bisecting_kmeans	5	0.1458	4	58,013, 24,938, 24,890, 1,113
bisecting_kmeans	6	0.0642	5	29,754, 28,259, 24,938, 24,890, ...
bisecting_kmeans	7	0.0430	6	29,754, 28,259, 24,890, 12,549, ...
bisecting_kmeans	8	0.0657	7	29,754, 28,259, 14,560, 12,549, ...
gaussian_mixture	2	0.1132	2	65,243, 43,711
gaussian_mixture	3	0.2658	3	82,004, 21,255, 5,695
gaussian_mixture	4	0.0711	4	53,505, 40,067, 13,447, 1,935
gaussian_mixture	5	0.0778	5	44,856, 44,575, 17,160, 1,802, ...
gaussian_mixture	6	0.1288	6	63,723, 39,033, 3,439, 2,311, ...
gaussian_mixture	7	0.1474	7	60,778, 21,684, 19,969, 5,645, ...
gaussian_mixture	8	0.0724	8	38,418, 34,464, 19,578, 13,202, ...

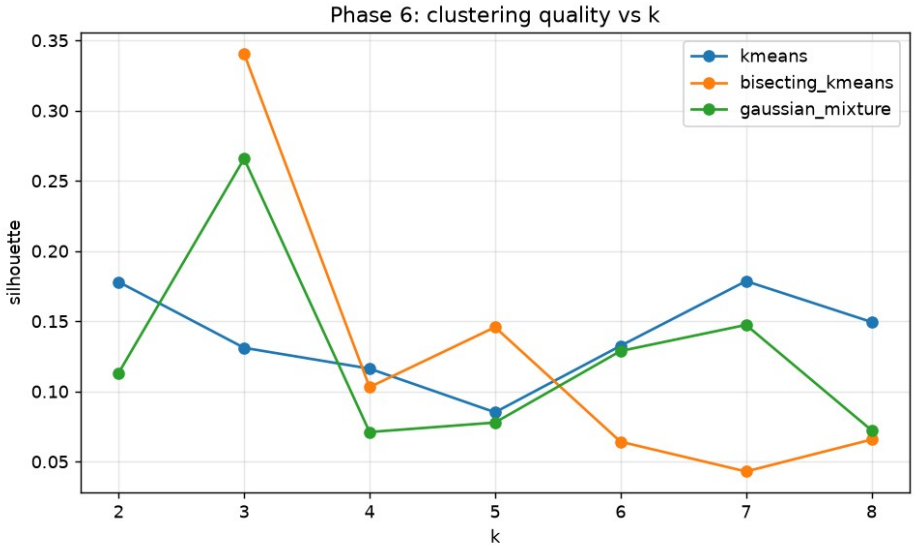


Figure 4. Silhouette against k for all three algorithms. Degenerate fits are omitted from the curves. No algorithm exceeds 0.35, and the surface is non-monotonic and unstable.

The best configuration is **bisecting_kmeans_k3** at silhouette **0.3406**. Two qualifications are essential. First, although k = 3 was requested, the fit returned only **2 non-empty clusters**; at k = 2 the same algorithm degenerated to a single cluster and was skipped. Second, a silhouette of 0.3406 is weak in absolute terms. Editor behaviour is better described as a continuum than as a set of well-separated groups, so the partition below should be read as a descriptive summary rather than as discovered natural classes.

Cluster	Editors	Mean edits	Gap CV	Minor ratio	Revert ratio	Edits/active day	Bot share	Bot enrichment
0	82,951	13	1.73	0.03	0.396	7.1	0.0007	0.19x
1	26,003	344	3.66	0.21	0.166	10.5	0.0132	3.58x

The two clusters separate cleanly on activity: cluster 0 contains 82,951 low-volume accounts averaging 13 edits with a high revert ratio of 0.396, consistent with casual and inexperienced editing. Cluster 1 contains 26,003 high-volume accounts averaging 344 edits, a far higher minor-edit ratio (0.21 against 0.03) and a much lower revert ratio, consistent with experienced maintenance work.

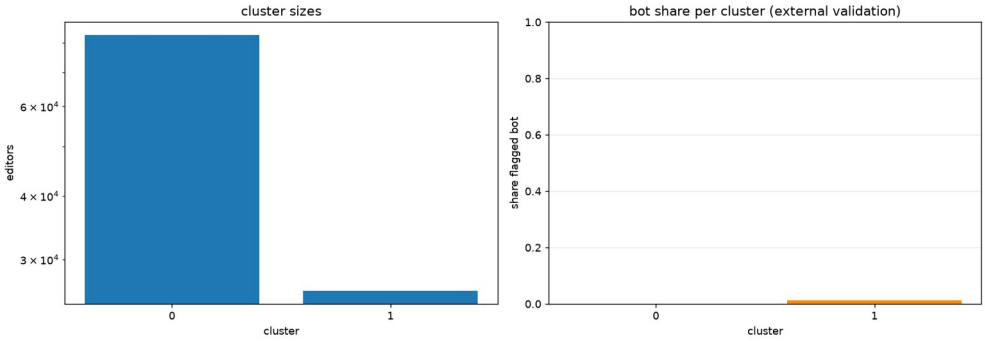


Figure 5. Cluster sizes on a logarithmic scale (left) and the share of each cluster flagged as automated (right). Automated accounts are a small minority everywhere.

8.4 External validation, and a metric that does not work

H3 is not supported, and the reason is instructive. The obvious validation metric for a partition against a held-out binary flag is purity, and ours is 0.9963, which appears excellent. It is in fact meaningless. Automated accounts are only 0.37% of **accounts** — 401 of 108,954 — because bots are a small number of accounts making a very large number of edits. The majority-class baseline is therefore 0.9963, and our purity exceeds it by 0.00000. The clustering supplies literally no information about automation by this measure, and reporting the figure without its baseline would have been straightforwardly misleading.

The informative measure is enrichment: the bot share of a cluster divided by the overall bot share. Automated accounts are **3.58 times over-represented** in the high-activity cluster and 0.19 times under-represented in the low-activity one. That is a real effect and in the expected direction, but it is modest, and it is explained adequately by bots being high-volume editors rather than by the clustering having identified automation as such.

The pipeline was amended to report purity only alongside its own baseline, and to emit an explicit warning when the lift falls below one percentage point.

9. Discussion

9.1 Outcome of the stated hypotheses

#	Hypothesis	Outcome
H1	Cadence regularity, not volume, is the strongest automation signal	**Partly supported.** Cadence features contribute materially, but the tree ensembles distribute importance across cadence, namespace and edit-size features rather than relying on any one. The clean single-feature story the literature suggests did not materialise.
H2	Structural features alone capture most achievable performance	**Supported, more strongly than expected.** GBT reaches ROC-AUC 0.9756 with no comment vocabulary; adding text slightly lowers it.
H3	Clustering will partially recover the automation boundary	**Not supported.** The partition separates activity level, not automation. Enrichment is only 3.6x and purity equals its baseline exactly.

H3 failing is the more interesting of the three results, and it is retained here rather than quietly dropped. The two tasks together make a coherent point: automation is highly learnable from behaviour *when supervision is available*, reaching ROC-AUC 0.9759, yet it is not the dominant axis along which editor behaviour naturally varies. What dominates unsupervised variation is simply how much an account does. Supervised and unsupervised views of the same feature space answer different questions, and conflating them would have produced a false claim.

9.2 Scalability

Spark is not justified here by the raw size of the input — a determined analyst could force 951 MB through a single machine's memory. It is justified by the shape of the workload. The per-editor aggregation in Phase 6 groups eleven million revisions by actor and computes windowed inter-edit intervals, which is a sort-merge shuffle whose working set scales with the number of distinct actors rather than with available memory. Spark spills to disk and completes; an in-memory group-by either thrashes or fails. Equally, the same code scales to the full corpus unchanged: simplewiki is one wiki of roughly 900, and redirecting the configuration at enwiki moves the input from about 1 GB to hundreds of GB across monthly shards with no rewrite.

Pipeline stage	Parallelisation mechanism
Raw TSV read	bzip2 is splittable under Hadoop, so the file was read by 12 concurrent tasks. A gzip dump would have been a single task — a real design trap
Column projection	Parquet retains 31 of 78 columns, so later phases read roughly a third of the bytes via columnar pruning
Label partitioning	The prepared dataset is partitioned by label_bot, allowing class-conditional queries to prune whole directories
Feature engineering	Entirely row-local withColumn expressions: embarrassingly parallel, no shuffle
Editor aggregation	groupBy plus a window lag, producing a sort-merge shuffle across 48 partitions. The dominant cost
Truncated SVD	Distributed RowMatrix; only the small right singular matrix is collected and broadcast
Model training	MLlib broadcasts the model and reduces gradient or histogram statistics per partition

Adaptive Query Execution is enabled, so post-shuffle partition counts are coalesced at run time rather than guessed in advance.

9.3 Challenges for very large datasets

- **Actor skew is the dominant risk.** Edits per account follow a heavy power law: a handful of bots have millions of revisions while the median account has one. A naive groupBy on the actor key sends one enormous partition to a single task, which straggles or exhausts memory while the remaining cores idle. Mitigations, in order of preference: adaptive skew-join handling (enabled); salting the heaviest keys followed by a second reduction; and bounded-memory sketches in place of exact approximate-percentile computation over a full partition.
- **The window function is the stage to optimise first.** The lag over a partition ordered by timestamp requires a full sort within each partition. At enwiki scale the right move is to pre-partition the Parquet by actor hash at write time so that the sort becomes local.
- **Small-files problem.** Writing one Parquet file per task across hundreds of monthly shards would produce tens of thousands of tiny files and cripple the subsequent read. We repartition before writing; at larger scale this becomes an explicit compaction step.
- **Broadcast ceiling.** Broadcasting the right singular matrix is trivial at 2048 by 24. Raising the TF-IDF dimension to, say, 2^20 would make it too large to broadcast and force a distributed matrix multiplication instead.
- **Sampling must follow the split.** Any stratified sampling to address class imbalance has to happen after the actor-disjoint split, or the disjointness guarantee is silently broken.
- **Regime mixing.** The all-time snapshot spans 2001 to 2026. Editing norms, tooling and bot policy changed enormously across that period, so a single model implicitly averages several distinct regimes.

At larger scale the appropriate response is to restrict to a recent window, or to treat era as an explicit stratum.

9.4 Where the model is weak

- **The label covers only flagged accounts.** Unapproved or unregistered automation is a false negative by construction, so the reported recall is measured against an incomplete positive class. The true recall against *all* automation is unknown and is certainly lower.
- **Semi-automated tooling has no clean boundary.** Tools such as AutoWikiBrowser and Huggle produce machine-speed edits under human direction. These accounts genuinely occupy the space between the classes, and no decision boundary can be correct for them in principle.
- **Default thresholding is not optimal.** Random forest on the structural set shows strong ranking quality (PR-AUC 0.8948) alongside poor bot-class F1 (0.7057) at the 0.5 threshold. This is a thresholding artefact, not a modelling failure; a deployment would select the threshold from the precision-recall curve against an explicit reviewer-capacity budget.
- **Single-wiki scope.** simplewiki is smaller, and its bot population more concentrated, than enwiki. The pipeline generalises; the specific numbers do not.

10. Conclusion

We built and ran a complete Spark MLlib pipeline over 10,987,077 real Wikimedia edits, covering ingestion of a 951 MB compressed dump into a columnar store, DataFrame-based cleaning under an enforced null contract, 35 engineered behavioural features together with a 24-dimension truncated SVD of comment text, and a comparison of three classifiers against three clustering algorithms, all evaluated under an actor-disjoint split that guarantees no test account was seen during training.

Automated editing is highly predictable from behaviour alone: the best classifier attains ROC-AUC 0.9759 and PR-AUC 0.9326 on unseen accounts, with bot-class precision 0.9806. The paired ablation establishes the more useful finding — structural behaviour is essentially sufficient, since gradient-boosted trees reach ROC-AUC 0.9756 without reading a single comment token. Unsupervised clustering of the same editors, by contrast, recovers activity level rather than automation, with bots only 3.6 times enriched in the busiest cluster.

Two methodological points generalise beyond this dataset. First, label validity deserves examination before any modelling: our reason for choosing Wikimedia over GDELT was that GDELT's labels are deterministic functions of a feature column, which would have made a strong-looking result vacuous. Second, a validation metric must be read against its own baseline: our cluster purity of 0.9963 looked like a success and was in fact identical to the majority-class rate. Both are failures that produce impressive numbers, which is precisely what makes them worth guarding against.

10.1 Future scope

- Scale to enwiki monthly shards on a multi-node cluster and measure realised speed-up, rather than arguing scalability from stage behaviour in local mode.
- Score the live EventStreams feed with the trained model using Structured Streaming, turning the batch classifier into an operational triage filter.
- Model label_reverted as a second supervised task; it is already derived and carried through the pipeline.
- Weight the clustering by edit volume, or cluster within activity strata, to test whether automation becomes a visible axis once the activity gradient is controlled for.
- Select the decision threshold against an explicit reviewer-capacity budget instead of reporting metrics at 0.5.
- Treat editing era as a stratum or feature, so the model is not averaging across two decades of changing norms.

11. References

[1] Wikimedia Foundation. MediaWiki History Dumps, snapshot 2026-10. https://dumps.wikimedia.org/other/mediawiki_history/ (CC0).

[2] Wikimedia Foundation. EventStreams. https://stream.wikimedia.org/v2/stream/recentchange

[3] Wikimedia Foundation. Analytics/Data Lake/Edits/MediaWiki history. https://wikitech.wikimedia.org/

[4] A. Halfaker and D. Taraborelli. "ORES: Scaling up machine learning-driven quality control in Wikipedia." Wikimedia Research, 2015.

[5] M. Potthast, B. Stein and R. Gerling. "Automatic Vandalism Detection in Wikipedia." In Proc. ECIR, 2008, pp. 663-668.

[6] B. T. Adler, L. de Alfaro, S. M. Mola-Velasco, P. Rosso and A. G. West. "Wikipedia Vandalism Detection: Combining Natural Language, Metadata and Reputation Features." In Proc. CICLing, 2011, pp. 277-288.

[7] C. A. Davis, O. Varol, E. Ferrara, A. Flammini and F. Menczer. "BotOrNot: A System to Evaluate Social Bots." In Proc. WWW (Companion), 2016, pp. 273-274.

[8] Z. Chu, S. Gianvecchio, H. Wang and S. Jajodia. "Detecting Automation of Twitter Accounts: Are You a Human, Bot, or Cyborg?" IEEE Transactions on Dependable and Secure Computing, vol. 9, no. 6, 2012, pp. 811-824.

[9] H. T. Welser, D. Cosley, G. Kossinets, A. Lin, F. Dokshin, G. Gay and M. Smith. "Finding Social Roles in Wikipedia." In Proc. iConference, 2011, pp. 122-129.

[10] D. Yang, A. Halfaker, R. Kraut and E. Hovy. "Who Does What: Editor Role Identification in Wikipedia." In Proc. ICWSM, 2016.

[11] S. Kumar, F. Spezzano and V. S. Subrahmanian. "VEWS: A Wikipedia Vandal Early Warning System." In Proc. KDD, 2015, pp. 607-616.

[12] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker and I. Stoica. "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing." In Proc. NSDI, 2012.

[13] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, et al. "MLlib: Machine Learning in Apache Spark." Journal of Machine Learning Research, vol. 17, no. 34, 2016, pp. 1-7.

[14] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, et al. "Spark SQL: Relational Data Processing in Spark." In Proc. SIGMOD, 2015, pp. 1383-1394.

[15] Apache Software Foundation. Apache Spark 3.5.3 MLlib Guide. https://spark.apache.org/docs/3.5.3/ml-guide.html

[16] J. Dean and S. Ghemawat. "MapReduce: Simplified Data Processing on Large Clusters." In Proc. OSDI, 2004.